Department of Computer Science

Assignment 2 - CS1813 Software Design
Coursework for CS1704 Group Project 2024/25

Provisional Version

TABLE OF CONTENTS

Assessment Title	Software Design
Module Leader	Dr Yasoda Jayaweera
Distribution Date	21/11/2025
Submission Deadline	06/02/2026 at 11:00 GMT
Feedback by	27/03/2026
Contribution to overall module assessment	100%
Indicative student time working on assessment	90 Hours
Word or Page Limit (if applicable)	30 Pages (not including cover page, table of contents, references)
Assessment Type (individual or group)	Individual

MAIN OBJECTIVE OF THE ASSESSMENT

This assignment is used to assess your abilities to design a computer program as a solution to a problem, and your ability to communicate your solution.

This assignment contributes 100% to the grade in CS1813 Software Design. It is assessed by your tutor.

DESCRIPTION OF THE ASSESSMENT

Each group member must choose **one** unique SwiftBot task described in Appendix A and design an algorithm for it. You will also implement the **same** task for Assignment 3. There are 11 tasks in total, and **no two members may select the same task**. Therefore, it is recommended that you get together with your group and collectively choose a task. Once you have decided on a task, you must inform your tutor.

The task descriptions specify the required core functionalities of the program. However, higher grades will be awarded for additional features that extend this core. These additional functionalities **must not replace or conflict** with the required requirements. They should be substantial, improve the usability of your program, and clearly demonstrate your software development skills.

Brunel University of London

The software design should be included in a report. The report must include all the following sections:

1. **Software Requirements Specification (SRS)**
   In this section, you should present a Software Requirements Specification (SRS) for the SwiftBot task you have chosen.

   First, you must generate an initial requirements specification using a Large Language Model (LLM) of your choice that is available under a free subscription.

   Next, you must refine and improve this initial requirements specification so that it aligns with the requirements specification structure and the quality characteristics discussed in the module. The **improved SRS should clearly outline all required functionalities, non-functional requirements, and any additional functionalities** you choose to include for your selected task.

   Your answer to this section must include the following:
   a) Name of the LLM used, model version, date you accessed it and the prompt you used
   b) The requirements specification generated by the LLM
   c) A description of the improvements you made to the initial SRS (in section b). (page limit: no more than 1 page)
   d) The improved requirements specification

   Note: For theory and examples of writing a good requirement specification, refer to the CS1704 Week 3 lecture on 'User Requirements' and the 'Week 3 - Writing User Requirements' laboratory worksheet and sample answer.

2. **Algorithm Design**
   In this section, you should present the algorithm design for the SwiftBot task you have chosen. The algorithm should describe how the program will function. The algorithm must be represented using flowcharts.

   You must first generate an initial representation of your algorithm using a Large Language Model (LLM) that is available under a free subscription.

   Next, you must refine and enhance this initial design to ensure it aligns with the flowchart notation and the quality characteristics covered in the module. The **improved design must include all required functionalities and any additional functionalities you have chosen to add for your selected task**.

   Given the complexity of the task, you may need multiple flowcharts to present the algorithm with the required granularity. You should use separate sub-processes to clearly indicate how you will validate the input, process the data, and produce the output, as well as a main algorithm that integrates these sub-processes into one workable solution. As a final results, you should have **at least three** (probably more) separated **flowcharts** to describe your proposed solution.

   Your answer must include the following:

a) Name of the LLM used, model version, date you accessed it and the prompt you used
b) The flowchart design generated by the LLM
c) A description of the improvements you made to the initial design (in section b) (page limit: no more than 1 page)
d) The improved algorithm design

Note: For theory and examples of designing algorithms using flowcharts refer to the CS1704 Weeks 4, 5 and 11 lectures on 'Algorithm Design' and the Week 4 'Designing Algorithms using Flowcharts' laboratory worksheet and sample answers.

3. **User Interface Design**
In this section, you should present the user interface (UI) design for the task you have chosen. The UI should show what the user will see while interacting with the program. All tasks must be implemented using a Command-Line Interface (CLI). Therefore, your UI design should illustrate how the interaction will appear in a **command-line environment**.

First, you should generate an initial UI design using a Large Language Model (LLM) that is available under a free subscription.

Next, you must refine and improve this initial design based on the principles discussed in the module. The **improved design must cover all required functionalities and any additional functionalities you have chosen to add for your selected task**. You may include all unique screens of the CLI, along with representative variants for key error messages and system states, to clearly demonstrate how the interface behaves without documenting every minor variation.

Your answer must include the following:
a) Name of the LLM used, model version, date you accessed it and the prompt you used
b) The UI prototype generated by the LLM
c) A description of the improvements you made to the initial design (in section b) (page limit: no more than 1 page)
d) The improved UI design

Note: There is **no implementation requirement at this stage**. What is required is a set of images showing the UI design, not Java code.

4. **Planning and Monitoring Progress** (page limit: no more than 2 pages)
In this section, you must produce **a Gantt chart.** Your plan must **cover both the design and implementation stages of the SwiftBot task**. The Gantt chart should show how you planned, managed, and tracked the progress of your work for the design assignment, as well as how you will plan for the completion of the implementation assignment.

Therefore, you are expected to read and understand the requirements in the Implementation Assignment Brief before creating your schedule.

The Gantt chart must use a **weekly timeline as a minimum**. It should include a breakdown of tasks into manageable activities, along with a clear list of deliverables and deadlines.

You are expected to **show** your ongoing work for the design assignment (in **incremental and iterative drafts**) to your tutor during the bi-weekly tutorial meetings and obtain feedback (see the study guide for the tutorial meeting schedule). Consequently, your project plan and **progress must be verifiable by your tutor**.

5. **References**

Harvard referencing must be used if external sources such as websites, articles or books were used in composing answers. For example, if you have included additional features inspired by existing technologies/ solutions, include references to your sources of inspiration.

Note: You may refer the following link to get more information about Harvard referencing style. https://libguides.brunel.ac.uk/referencing/overview

If Generative AI tools are used in any sections other than Sections 1(b), 2(b), 3(b), and 4(b), you are required to provide references. For each instance, you must clearly specify the AI tool used, the model version, the exact prompt submitted, and the output generated by the tool.

LEARNING OUTCOMES AND MARKING CRITERIA

Learning Outcomes

In order to get a pass grade (D- or above) in this assignment, you must meet the learning outcomes below, that is, you must demonstrate ability to:

LO3: Effectively present and communicate solutions.
LO4: Take an open-ended non-trivial problem and define and refine the requirements.
LO5: Plan, manage and track a non-trivial activity.
LO6: Create and use technical documentation.

The marking criteria are provided in the table below.

Grade Descriptor	Marker discretion to apply +/- grades
One or more of the following criteria may apply: ▪ The design (SRS, algorithm and UI) is completed for the incorrect task. ▪ Little to no improvements have been made to the LLM-generated output, or the improvements are difficult to understand, as key sections are unclear, incomplete, or missing. ▪ The design does not cover all required functionalities. ▪ The report does not follow the specified structure, or is missing one or more required sections.	E/ F grade
The following criteria must all be met: ▪ The improved SRS lists all required functionalities stated or implied in the task description as a numbered list. ▪ The improved algorithm design covers all required functionalities stated or implied in the task description. The algorithm includes at least two sub-processes integrated into a main algorithm. The logical flow is clear and correct, and correct flowchart notation is used throughout. ▪ The improved UI design covers all required functionalities stated or implied in the task description. Each UI screen includes a short description and a clear mapping to the requirement(s) it addresses.	D-grade (D-, D, D+)
All the criteria for a D-grade plus: ▪ In the improved SRS, each requirement clearly specifies responsibility (system behaviour or user actions). Requirements are presented in a logical order and use consistent terminology throughout. ▪ The improved algorithm design uses consistent styling, formatting and is fully implementation-free. ▪ The improved UI design is well-organised, with appropriate spacing, clear headings and a consistent layout and style throughout.	C-grade (C-, C, C+)
All the criteria for a C-grade plus: ▪ In the improved SRS, requirements are specific and sufficiently detailed that two independent developers would produce the same implementation. ▪ In the improved algorithm design, the data flow between steps and sub-processes is clear and accurate, making it easy to trace how data is transformed throughout the algorithm. Appropriate algorithmic data structures and meaningful names are used. ▪ The improved UI design is preventive, capturing all foreseeable errors and providing informative error messages.	B-grade (B-, B, B+)
All the criteria for a B-grade plus: ▪ The improved SRS identifies the non-functional requirements applicable to the task. Proposed additional functionalities are	A-grade (A-, A, A+)

substantial and modelled into a complete, coherent set of requirements. ▪ In the improved algorithm design, each step is sufficiently granular that two independent developers would produce nearly identical implementations based on the design. The solution shows effective use of sub-processes, with each sub-process having a clear, self-contained purpose. ▪ The improved UI design incorporates elements such as ASCII art and ASCII headers to enhance visual appeal and create a more engaging interface. ▪ There is evidence that the plan for the design task was followed, with at least one draft of the design presented during tutorial meetings.	
All the criteria for a A-grade plus: ▪ The proposed additional functionalities are fully integrated into the improved algorithm design. The overall algorithm is complete, coherent, and free from unnecessary complexity. ▪ The overall report quality is excellent with a cover page, table of contents, section numbering, page numbers, appropriate use of academic language and a well-organised layout of information. ▪ Produces a well-thought-out, clear and risk-aware plan for the implementation phase, including a list of deliverables and associated deadlines.	A*

FORMAT OF THE ASSESSMENT

The report should be submitted as a single PDF file. **The report must include the following sections:**

1. Software Requirements Specification
2. Algorithm Design
3. User Interface Design
4. Planning and Monitoring Progress
5. References

The main text of the report (excluding cover page, table of contents, references) should not exceed 30 pages, using a minimum font size of 11 pt. As a general guideline, the improved answers for Sections 1 - 3 and Section 4 should take up to 15 pages, with the remaining pages allocated to subsections (a), (b), and (c) within Sections 1 - 3.

This assignment element is individually assessed. Your tutor will provide formal feedback on your work and assign a grade on WISEflow in week 26 (27/03/2026).

IMPORTANT: Prior to submission, you are expected to show your work (as in incremental and iterative drafts) to your tutor during bi-weekly meetings to get feedback. If your solution has not been satisfactorily demonstrated to your tutor, or if there are concerns about the authenticity of your work, you will be invited to a viva voce examination to confirm you are its author. If you fail to demonstrate a thorough understanding of the submitted work, an academic misconduct case will be raised.

Submission Instructions

You must submit your coursework as a PDF file on WISEflow by 06/02/2026 at 11:00 GMT. You can follow the link to WISEflow through the module's section on Brightspace or login in directly at https://uk.wiseflow.net/brunel. The name of your file should follow the normal convention and must therefore include your student ID number (e.g., 0612345.pdf). It can also include the module/assessment block code (e.g., CS1813_0612345.pdf).

IMPORTANT: Any links to diagrams, files, or other materials stored on cloud platforms or online repositories will **not** be accepted as a valid part of your submission. All required content must be included directly within the report itself.

Avoiding Academic Misconduct

Before working on and then submitting your coursework, please ensure that you understand the meaning of plagiarism, collusion, and cheating (including contract cheating) and the seriousness of these offences. Academic misconduct is serious and being found guilty of it results in penalties that can reduce the class of your degree and may lead to you being expelled from the University. Information on what constitutes academic misconduct and the potential consequences for students can be found in Senate Regulation 6.

You may also find it useful to read this PowerPoint presentation which explains, in plain English, the different kinds of misconduct, how to avoid (even accidently) committing them, how we detect misconduct, and the common reasons that students give for engaging in such activities.

If you are experiencing difficulties with any part of your studies, remember there is always help available:

- Speak to your personal tutor. If you're not sure who your tutor is, you can find the information on eVision or you can ask at the Student Hub (studenthub@brunel.ac.uk).
- The Student Hub can also provide advice on support and welfare issues.

Expectations of Artificial Intelligence Use

The University has general guidance on using artificial intelligence in your studies.

Using AI-generated content and presenting it as your original work in any form of assessment, be it formative or summative, is strictly prohibited.

Despite Generative AI offers many useful services such as information search and retrieval, concept explanations, grammar and writing enhancements and coding assistance, it is crucial to recognise its inherent limitations. For instance, responses generated by AI can be overly general, inaccurate, biased, or even fabricated. They often lack proper references and detailed insights and they may pose challenges in terms of intellectual property rights and data privacy. Misuse of Generative AI can lead to academic misconduct.

Should you have used any information generated by AI in your work, you must provide clear references to the AI tools used. To maintain the authenticity of your work, it is essential to keep regular communication with your tutor and consistently update them on your progress. In the event of any concerns regarding the integrity of your work, a viva voce examination may be scheduled for further evaluation.

LATE COURSEWORK

The clear expectation is that you will submit your coursework by the submission deadline stated in the study guide. In line with the University's Coursework Submission Policy (revised in January 2025), coursework submitted up to 48 hours late will be accepted but the penalties set out in the 'Late Submission' section of the policy will be applied. Work submitted over 48 hours after the stated deadline will automatically be graded NS (non-submission).

Please refer to the Computer Science student information pages and the Coursework Submission Procedure pages for information on submitting late work, penalties applied and procedures in the case of Extenuating circumstances.

APPENDIX A – SWIFTBOT ASSESSMENT TASKS

Task 1: Master Mind

The purpose of this program is to implement a Mastermind game where a user plays against the SwiftBot, using the SwiftBot's camera to read colour inputs.

The game offers two modes: Default and Customized. At program start, users select their preferred mode by pressing either button 'A' for Default mode or button 'B' for Customized mode.

Default mode: At the start of the program, the computer should generate a random code of four colours from a range of six different colours [Red (R), Green (G), Blue (B), Yellow (Y), Orange (O), Pink(P)]. There should not be any repeating colours in the generated code. The player should be able to enter a guess, for example 'RGBY', using colour cards. To enter the user's answer, the program should prompt the user to hold the colour cards, one after the other, in the desired order. Then the SwiftBot will take a photo of each card and determine the colour code. For example, the SwiftBot prompts the user to hold the first card in front of the camera and then takes a photo of it. Next it will prompt the user to hold the second colour card and takes a photo. This should continue until all the four colour cards are scanned.

The program should process the images one by one to determine the colour sequence. For example, your program could convert the image into a pixel matrix and then use the RGB values of each pixel to compute the average colour value. This average colour value could then be used to determine the colour of the card being scanned.

After each guess, the program provides feedback using '+' and '-' symbols. The program should print '+' if the colour occurs in the code and is in the right position, and '-' if the guessed colour is in the code but is in the wrong position.

Any '+' symbols should be displayed first, and, the SwiftBot does not reveal which colour in the guess was right. For example, for code 'RGBY' and the player's guess 'GPBY', the SwiftBot should display: ++-

A payer has 6 attempts to guess the code correctly. If the player runs out of guesses, the SwiftBot should reveal the code and declare the game lost. If the player enters the correct code, they win the game, and the SwiftBot should print a congratulatory message. The program should display a player-vs-computer score; for example, if the player has won a game, the score is 1-0.

After each game, players can continue the game of quit. If the player wants to quit, they need to press button 'X', and if the player wishes to continue, button 'Y' should be pressed. If the player continues playing, the score should be updated after each game, accordingly. The program should log the details of each round: round number, computer's code, player's input, score, total number of guesses and guesses left, to a text file. Before terminating, the program should display the log file's location to the user.

Customized mode: At the start of the game, the player should be able to choose the number of colours in the code (3 - 6), and the maximum number of guesses allowed.

This program should be implemented using a command-line user interface (input and output printed in the command prompt). Your program should make appropriate error checks and exception handling in order to ensure that the input values are within the valid range specified in the task description. Invalid input should be rejected, and informative error messages should be displayed, prompting the user for new input. The software should be designed to enhance user experience and ease of use.

While the description above outline the required core functionalities, higher grades will be awarded for additional features. These additional functionalities **must not replace or conflict** with the required functionalities of the task. They should be substantial, improve the usability of your program and clearly demonstrate your software development skills.

A substantial additional functionality should:

- accept an input from the user,
- perform meaningful processing based on that input and
- produce an output directly related to the SwiftBot's movement or behaviour.

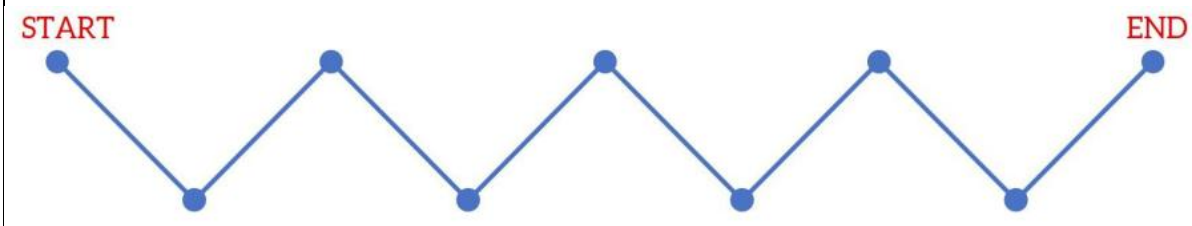
You may include as many additional functionalities as you wish, however, implementing one substantial additional functionality is sufficient to achieve an A* grade. You may find it useful to discuss this with your tutor.

Task 2: Zigzag

The purpose of this program is to make the SwiftBot move in a regular zigzag fashion.

Although it is possible to attach a pencil or marker to the SwiftBot, preferably close to the battery, to get a decent-looking drawing, it is not necessary; if the movement can be eyeballed, that should be sufficient.

For this task, we will assume that a zigzag path consists of lines of equal length that are orthogonal to each other. When the program starts, the user should be prompted to scan a QR code containing two values: the length of a zigzag section in centimetres and the number of zigzag sections (all sections are of the same length). For example, the zigzag in the illustration below consists of 8 zigzag sections.



The user's input should be encoded as follows: Value1-Value2. The length of a zigzag section (Value1) should be at least 15 centimetres and no more than 85 centimetres. The number of zigzag sections (Value2) should be even and should not exceed 12. For example,

'20-6' should be interpreted as a command to draw a zigzag path consisting of 6 zigzag sections, each with a length of 20 cm.

The speed of the wheels of the robot should be randomly generated. Given that the user provides the length of each zig zag section in centimetres (i.e. travel distance), the program needs to calculate the time (in seconds/milliseconds) the robot should move to cover the user-provided distance. You can calculate the time that the robot should move for by experimenting with your robot and using simple mathematics (refer Calibrate Your SwiftBot worksheet for more guidance).

Once the length and the number of zigzag sections have been given, the program should display the user inputs and randomly generated speed, then the SwiftBot should set the underlights to green and start moving forward until it moves for the given zigzag section length. The SwiftBot should then stop for one second, turn orthogonally to the path travelled and start moving in the new direction for the length of the second zigzag line. As it is traversing the second section, its LEDs should change to blue. At the third section, it should change its LED back to green (like in the first section of the zigzag). In the fourth section, the SwiftBot should go back to blue, and so on.

Once the SwiftBot has completed traversing the given number of zigzag sections, it should turn around and retrace its movements until it reaches the original starting position (displaying the same LED colour as when it first traversed each section). When the SwiftBot reaches the initial starting position, it should stop and turn off the LEDs. Next, it should write the following information to a text file: the user inputs (that is, length of each zig zag section and number of zig zag sections), the randomly generated wheel speed, the total length of the traversed path (start to the end position of the zigzag, but not including travelling back), the duration (in seconds) that the robot took to complete the move from START to END, and the straight line distance travelled from the START to the END of the zigzag.

Then it should prompt the user to scan another QR code or terminate the program. If the user wants to scan another QR code, they should be prompted to press button 'Y'. The program should terminate when button 'X' is pressed. Before terminating the program, it should display the following to the user: the number of zig zag journeys completed, details of the journey that resulted in the longest straight line distance (user inputs and straight line distance), details of the journey that resulted in the shortest straight line distance (user inputs and straight line distance), and the location of the log file.

This program should be implemented using a command-line user interface (input and output printed in the command prompt). Your program should make appropriate error checks and exception handling in order to ensure that the input values are within the valid range specified in the task description. Invalid input should be rejected, and informative error messages should be displayed, prompting the user for new input. The software should be designed to enhance user experience and ease of use.

While the description above outline the required core functionalities, higher grades will be awarded for additional features. These additional functionalities **must not replace or conflict** with the required functionalities of the task. They should be substantial, improve the usability of your program and clearly demonstrate your software development skills.

A substantial additional functionality should:

- accept an input from the user,
- perform meaningful processing based on that input and
- produce an output directly related to the SwiftBot's movement or behaviour.

You may include as many additional functionalities as you wish, however, implementing one substantial additional functionality is sufficient to achieve an A* grade. You may find it useful to discuss this with your tutor.

For testing purposes, you may use any free QR code generator (such as https://qr.io) to create text-based QR codes.

Task 3: Snakes and ladders

The purpose of this program is to implement a Snakes and Ladders game that can be played by two players. The two players are the user and the SwiftBot. This task requires the robot to move along a predefined path (on the game board) and stop at specific locations (squares on the game board).

For this game, you will need a physical board, one playing piece for the user, and marker cards to indicate the positions of snakes' heads and tails, and ladder tops and bottoms. The game will use a six-sided die, and the rolling of the die is done by the program. Players (the user and the SwiftBot) should navigate their pieces from start to finish, avoiding the snakes and taking shortcuts up the ladders.

This version of Snakes and Ladders uses a 5 x 5 board. The board has 25 squares, starting from 1 and going up to 25. The size of one square is 25 cm x 25 cm. The overall size of the board is 125 cm x 125 cm. You will need to create your own board as per the dimensions mentioned above to test your program. During the examination, we will provide the board (and other contents) for you.

To start the program, the user should press button 'Y' on the SwiftBot.

At the start of the game, the program should prompt the user to enter their preferred name. The program may assign an appropriate name to the SwiftBot.

First, each player should roll the die, and the player who rolls the highest number will take the first turn. Rolling the die is done programmatically. When it is the user's turn to roll the die, the user should press button 'A' on the SwiftBot. Players start at square 1 and move their pieces. While the user moves their piece along the board, the SwiftBot should move to the correct square on the board. Each player rolls the die when their turn comes. Based on the dice value, the players move. The first player to reach square 25 wins the game. You should use a 2D matrix to simulate the physical board.

For example, if a player is on square 10 and they roll 5, they will move to square 15. A player wins if they arrive at/land on square 25, exactly. That is, if a player is on square 23, and they roll 4, they cannot move. They must keep playing until they roll exactly 2.

After each turn, the program should display the die value rolled and the players' movements (on the board) in the console. This could be a message as shown below or a visual representation of the board.

[Yasoda] rolled a [5] and moved from square [10] to [15].
[SwiftBot] rolled a [2] and moved from square [8] to [10].

Before the game begins, the program should place 2 snakes and 2 ladders on the board. Each snake has a head and a tail on the board. If a player lands on a square with a snake's head, they should move down to the square where the snake's tail ends. Similarly, if a player lands on a square with the bottom of a ladder, they should move up to the square where the ladder ends. The head of a snake is always in a square with a higher number than its tail, and the top of a ladder is always in a square with a higher number than its bottom. The head and tail of a snake, or the top and bottom of a ladder, cannot be in the same row. When placing ladders and snakes, you cannot place a snake's head or tail at either end of a ladder.

At the beginning of each game, the program should display to the user where the ladders and snakes are (i.e. the square where the start and end of each snake and ladder). Then you can prepare the physical board by placing the marker cards in the respective squares.

The game has two modes: Mode A or Mode B. Once the names are registered, the user should be prompted to select the mode. In Mode A, the game operates normally, and the square each player moves to is determined entirely by the value of the rolled die. In Mode B, the user is given a special privilege: when it is the SwiftBot's turn to move, the user can override the rolled dice value and choose a specific square they wish to move the SwiftBot to. The move can only be up to 5 squares at a given time. This privilege applies only to the user and not to the SwiftBot. The SwiftBot should move to the square the user instructs it to. All other rules, including movement, turn order, snakes, ladders, and winning conditions, remain unchanged in this mode.

The user should be able to quit the game at two points: when one of the players lands on square 5 and when one of the players lands on square 25. The 'X' button on the SwiftBot should be pressed to quit. Before terminating, the program should record the last position of both players, snakes and ladders on the board and current system date and time in the format yyyy.MM.dd.HH.mm.ss in a log file. The file path where the log file is saved should be displayed to the user before the program terminates.

To move along the grid, the SwiftBot should keep track of its current location and orientation (which direction it is facing and how it should move to reach the next square). You should find a suitable speed for the SwiftBot by experimenting with your robot (refer Calibrate Your SwiftBot worksheet for more guidance).

This program should be implemented using a command-line user interface (input and output printed in the command prompt). Your program should make appropriate error checks and exception handling in order to ensure that the input values are within the valid range specified in the task description. Invalid input should be rejected, and informative error messages should be displayed, prompting the user for new input. The software should be designed to enhance user experience and ease of use.

While the description above outline the required core functionalities, higher grades will be awarded for additional features. These additional functionalities **must not replace or conflict** with the required functionalities of the task. They should be substantial, improve the usability of your program and clearly demonstrate your software development skills.

A substantial additional functionality should:

- accept an input from the user,
- perform meaningful processing based on that input and
- produce an output directly related to the SwiftBot's movement or behaviour.

You may include as many additional functionalities as you wish, however, implementing one substantial additional functionality is sufficient to achieve an A* grade. You may find it useful to discuss this with your tutor.

Task 4: Traffic Light

The purpose of this program is to navigate the SwiftBot using colour codes detected by its camera.

The program should process images of traffic lights displaying three colours: red, green, and blue. The red and green traffic lights follow standard traffic conventions: red signals the SwiftBot to stop, while green indicates it should proceed forward. The blue light represents a special scenario where the SwiftBot must temporarily yield its path - similar to how vehicles give way to emergency vehicles in real-world situations. While actual lights can be used, coloured images may be substituted. The SwiftBot should respond to traffic lights when they are within 20 cm of its position.

The program should process the photo taken by the SwiftBot's camera to determine the colour of the traffic light. One recommended approach is to convert the image into a pixel matrix and use the RGB values of each pixel to compute an average colour value. This average colour value can then be used to determine the traffic light colour.

To start the program, the user must press button 'A' on the SwiftBot.

This should make the SwiftBot, first, turn the underlights to yellow, and, then, start moving forward at a low speed (initial speed) until it encounters the traffic light. The program should display the initial speed to the user. The SwiftBot should respond to the traffic light when it is 30 cm (or less) ahead of the robot. Your program should check for traffic lights at regular intervals while the SwiftBot is moving. Choose an appropriate time interval, according to your initial speed, to ensure reliable detection. When a traffic light is detected, the program should display both its colour and its distance from the SwiftBot.

Upon detecting a green light, the underlights should be set to green and the SwiftBot should pass the traffic light within 2 seconds. After passing the traffic light, the SwiftBot should stop for a second, set the underlights to yellow, and then the robot should move forward in its initial speed until it detects the next traffic light. The appropriate forward speed can be calculated through experimentation with your SwiftBot using simple mathematics (refer Calibrate Your SwiftBot worksheet for more guidance).

Upon detecting a red light, the SwiftBot should set its underlights to red and the SwiftBot should come to a stop at the traffic light. After a second, the SwiftBot should start moving forward at its initial speed until it detects the next traffic light.

Upon detecting a blue light, the SwiftBot should stop for a second, blink the underlights in blue and then, should turn left by 90 degrees. The robot should then move forward at a low speed for 1 second and stop. After a second, the SwiftBot should retrace its movement and move back to the original path. Then the robot should move forward at its initial speed until it detects the next traffic light.

The program terminates when the user presses the 'X' button on the SwiftBot. After every three lights detected (i.e., after the 3rd light, 6th light, 9th light, and so on) and after completing the navigation for the third light, the user should be prompted to either terminate the program or continue.

Before terminating, it should ask the user whether to display the execution log. If the user presses button 'Y', the program should display the following information: the number of times the SwiftBot encountered traffic lights, the most frequent traffic light colour the SwiftBot encountered, the number of times the most frequent traffic light was detected, and the total duration of execution.

If the user does not want to view the log, the user should press button ‘X’ again. In either case, the program should write the log information to a text file before terminating and display the location of the log file to the user before terminating.

This program should be implemented using a command-line user interface (input and output printed in the command prompt). Your program should make appropriate error checks and exception handling in order to ensure that the input values are within the valid range specified in the task description. Invalid input should be rejected, and informative error messages should be displayed, prompting the user for new input. The software should be designed to enhance user experience and ease of use.

While the description above outline the required core functionalities, higher grades will be awarded for additional features. These additional functionalities **must not replace or conflict** with the required functionalities of the task. They should be substantial, improve the usability of your program and clearly demonstrate your software development skills.

A substantial additional functionality should:

- accept an input from the user,
- perform meaningful processing based on that input and
- produce an output directly related to the SwiftBot’s movement or behaviour.

You may include as many additional functionalities as you wish, however, implementing one substantial additional functionality is sufficient to achieve an A* grade. You may find it useful to discuss this with your tutor.

Task 5: SpyBot

The purpose of this program is to transform your SwiftBot into a secure two-way communication device, capable of encoding and decoding messages using Morse code. Morse code encodes text by representing each character as a sequence of short and long signal units, known as dots and dashes. This program will simulate this pattern using the buttons and underlights on the SwiftBot to send and receive messages.

To give a bit of context to the task, an example scenario has been included to help you visualize the problem. Assume three agents, isolated in their safe houses, must find a way to communicate without breaking radio silence. The safe houses are arranged at the corners of a perfect equilateral triangle, two kilometres apart, too far to risk direct contact. Each house is assigned a name; 'A', 'B', and 'C'. Each house has access to a SwiftBot, which they plan to use as their communication device. Please note that at a given time only one SwiftBot is used to send and receive messages.

Before sending or receiving a message, each user is required to verify their identity to protect against unauthorized access. Each agent has been issued a unique QR code containing their call sign and location. The call sign is an alphanumeric string with no spaces or special characters. The location is a single character representing the house name ('A', 'B', or 'C'). The QR code encodes this information in the format <call sign>:<location>.

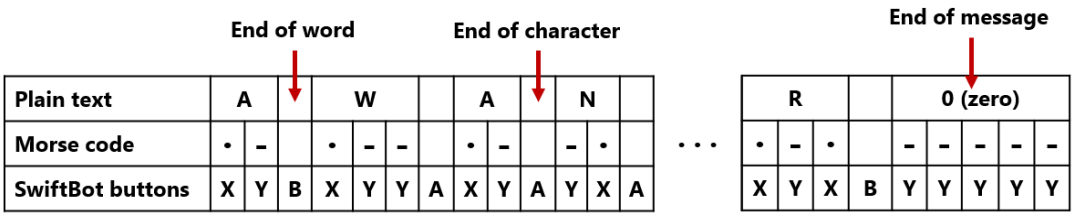
To convert a Morse code–encoded message to plain text and vice versa, you may use the Morse code dictionary stored as a plain text file. An example dictionary can be found in the Assessment folder on the CS1704 Brightspace page.

To send a message, the user must first authenticate themselves. Then, the user should provide the message in Morse code so the SwiftBot can record it. The following rules describe how to use the SwiftBot buttons to encode the message.

- Dot (.) → X button
- Dash (-) → Y button
- End of character → A button
- End of word → B button
- End of message → entre digit zero (0) in Morse code

When encoding a message to send, the message should always include the destination first and then the message that needs to be delivered. In this case, the destination name should be considered one word. For example, if the agent in safe house 'B' wants to send the message "want gear" to safe house 'A', this is how they should encode it using the SwiftBot buttons. Please note that this is a partially completed answer.

A *<end of word>* ***want*** *<end of word>* ***gear*** *<end of word>* *<end of message>*

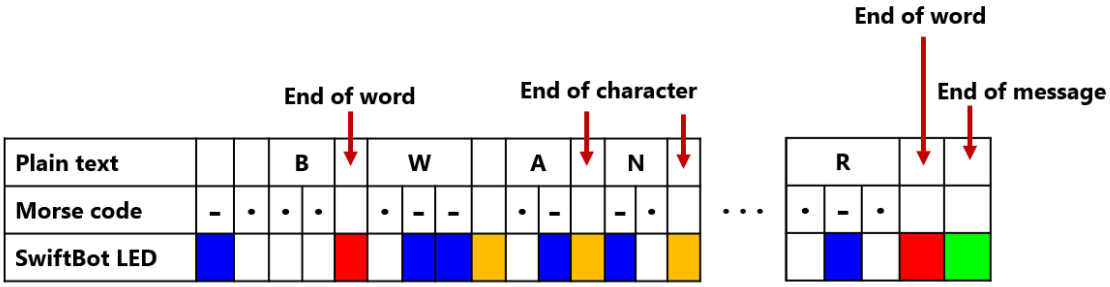


After storing the message, the SwiftBot should process and extract the destination. It should then travel to the specified destination. Upon reaching the destination, it should blink the LEDs in red to indicate it has a message to deliver. The SwiftBot should verify the receiver by prompting them to scan their QR code. Once verified, it should deliver the message by blinking the underlights according to the rules mentioned below.

- Dot → blink LEDs in White colour once
- Dash (-) → blink LEDs in blue colour once
- End of character → blink LEDs in Amber colour once
- End of word → blink LEDs in red colour once
- End of message → blink LEDs in green colour once

When delivering the message, the SwiftBot should first indicate the sender of the message. For example, when delivering the message 'want gear' to the agent in 'A' it should deliver in the following format and the LEDs will be lit as shown below. Please note that this is a partially completed answer.

B <end of word> want <end of word> gear <end of word> <end of message>



To demonstrate your solution meaningfully, you will run a scaled-down simulation at a 1:40 scale, so each side of the triangle measures 50 cm. Assume that your SwiftBot moves only along this triangular track. The locations and call signs of all agents must be stored in the robot. You must design your own unique call signs for each agent and use them in your solution and code. Ten seconds after delivering the message, the SwiftBot will return to its sender. The program should keep a log of all messages communicated in plain text including the details of sender and receiver, location, message and time the message was recorded and delivered.

To move along the grid, the SwiftBot should keep track of its current location and orientation (which direction it is facing and how it should move to reach the next square). You should find a suitable speed for the SwiftBot by experimenting with your robot (refer Calibrate Your SwiftBot worksheet for more guidance).

This program should be implemented using a command-line user interface (input and output printed in the command prompt). Your program should make appropriate error checks and exception handling in order to ensure that the input values are within the valid range specified in the task description. Invalid input should be rejected, and informative error messages should be displayed, prompting the user for new input. The software should be designed to enhance user experience and ease of use.

While the description above outline the required core functionalities, higher grades will be awarded for additional features. These additional functionalities **must not replace or conflict** with the required functionalities of the task. They should be substantial, improve the usability of your program and clearly demonstrate your software development skills.

A substantial additional functionality should:

- accept an input from the user,
- perform meaningful processing based on that input and
- produce an output directly related to the SwiftBot's movement or behaviour.

You may include as many additional functionalities as you wish, however, implementing one substantial additional functionality is sufficient to achieve an A* grade. You may find it useful to discuss this with your tutor.

For testing purposes, you may use any free QR code generator (such as https://qr.io) to create text-based QR codes.

Task 6: Draw Shape

The purpose of this program is to make your SwiftBot draw a shape. The SwiftBot should be able to draw two types of shapes namely squares and triangles.

Although it is possible to attach a pencil or marker to the SwiftBot, preferably close to the battery, to get a decent-looking drawing, it is not necessary; if the movement can be eyeballed, that should be sufficient.

When the program starts, it should prompt the user to scan a QR code containing information about the shape the robot should draw. The QR code should include the name of the shape and the respective side lengths in a specific pattern (explained below). The user should be able to terminate the program by pressing the 'X' button on the SwiftBot.

The shape should be encoded in the QR code as follows:
'S' to draw a Square
'T' to draw a Triangle

The codes for shapes are case insensitive. All shapes require additional input (i.e. lengths of each side) to be provided. For a single shape, the additional input(s) should be separated with a colon (" : ").

To draw a square, the input should also include the length of one side of the square. For example, 'S:16' should be interpreted as a command to draw a square of side length 16 cm. The length should be a positive integer number between 15 and 85 cm.

To draw a triangle, the length of the three sides of the triangle should also be included in the QR code. For example, 'T:16:30:24' should be interpreted as a command to draw a triangle of side lengths 16, 30 and 24. The lengths should be positive integer numbers between 15 and 85 cm. Once the length values have been entered, the program should also check whether the entered numbers for the length of the sides could form a triangle. Again, if a triangle cannot be formed with the given values, the program should let the user know. Before attempting to draw the triangle, the program should determine triangle angles.

Further, the program should be able to accept multiple shapes in one QR code. When multiple shapes are provided, the shapes should be separated by an ampersand (" & "). For example 'S:16&T:16:30:24' should be interpreted as an instruction to first draw a square of side length 16 cm, and then draw a triangle of side lengths 16, 30 and 24. When multiple commands are specified, the SwiftBot should execute them in sequence. A user can specify only up to 5 shapes using one QR code. When drawing multiple shapes, the SwiftBot should move 15 cm backwards from its starting position and start drawing the next shape.

Once valid values have been entered, the wheels of the SwiftBot should start moving at a low speed. Given that the length of the sides of the shape ('distance') is in centimetres, you need to calculate the seconds/milliseconds that the SwiftBot should move for – with that speed – in order to cover the distance that the user has input. You can calculate the time that the SwiftBot should move for by experimenting with your SwiftBot and using simple mathematics (refer Calibrate Your SwiftBot worksheet for more guidance).

Before drawing the shape, the program should display which shape will be drawn. When the drawing of a shape is completed the SwiftBot should stop and blink the underlights in green to indicate the drawing has completed. The program should carry on prompting the user to scan multiple QR codes to draw different shapes until the user presses 'X' button (on the SwiftBot) to terminate the program. When the 'X' button is pressed, the program should write the following information to a text file and display the file path where the log file is saved.

- The names and the sizes of all drawn shapes as well as the angles of the triangles and time taken to draw each shape, in the order they were drawn. For example: 'Square: 60 (time: 12000 seconds), Triangle: 60, 45, 75 (angles: 53.13, 36.87, 90; time: 15670 seconds), Square: 30 (time: 7100 seconds)
- The largest shape (by area) that was drawn and its size. For example: 'Square: 60'
- The shape (triangle or square) which was drawn most frequently, and how many times it was drawn. For example, 'Square: 2 times'.
- The average time it took to draw the shapes. For example, if the SwiftBot had drawn 3 shapes, which took 12000, 15670 and 7100 seconds, then in the file you should write: 11,590 seconds.

The program can then terminate.

This program should be implemented using a command-line user interface (input and output printed in the command prompt). Your program should make appropriate error

checks and exception handling in order to ensure that the input values are within the valid range specified in the task description. Invalid input should be rejected, and informative error messages should be displayed, prompting the user for new input. The software should be designed to enhance user experience and ease of use.

While the description above outline the required core functionalities, higher grades will be awarded for additional features. These additional functionalities **must not replace or conflict** with the required functionalities of the task. They should be substantial, improve the usability of your program and clearly demonstrate your software development skills.

A substantial additional functionality should:

- accept an input from the user,
- perform meaningful processing based on that input and
- produce an output directly related to the SwiftBot's movement or behaviour.

You may include as many additional functionalities as you wish, however, implementing one substantial additional functionality is sufficient to achieve an A* grade. You may find it useful to discuss this with your tutor.

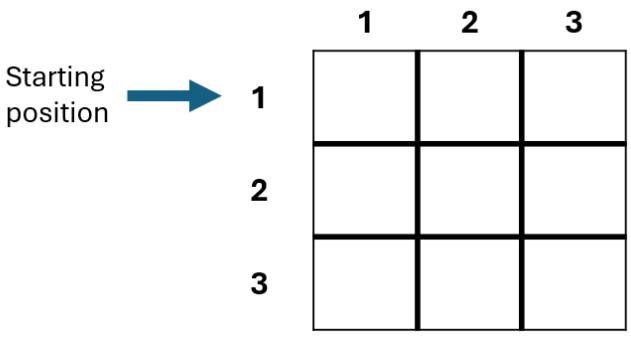
For testing purposes, you may use any free QR code generator (such as https://qr.io) to create text-based QR codes.

Task 7: Noughts and Crosses

The purpose of this program is to implement the Noughts and Crosses (also known as Tic-Tac-Toe) game that can be played between two players. The two players are the user and the SwiftBot.

For this game, you will need a physical board and four playing pieces labelled 'O' and 'X' for each player to mark their chosen locations.

Noughts and Crosses is played on a 3 × 3 board consisting of nine squares. Each square should measure 25 cm × 25 cm, giving the board an overall size of 75 cm × 75 cm. the figure below shows how the board squares are numbered. You are required to create your own board based on these dimensions to test your program. During the examination, the board and the playing pieces will be provided for you.



To start the program, the user should press button 'A' on the SwiftBot.

Each player must have a name. The program should prompt the user to enter their name using the keyboard. The SwiftBot can be assigned a suitable name by the program. Once both players' names are registered, the game can begin.

The game will use a six-sided die, and the rolling of the die is done by the program. First, each player should roll the die, and the player who rolls the highest number will take the first turn. The player who starts first will use the noughts ('O') pieces, and the other will use the crosses ('X') pieces. The user should specify their move using the keyboard in the format *[row_number,column_number]*.

When it is the SwiftBot's turn, the robot will select its move randomly and then it should move to the chosen square, blink its lights in green three times, and then return to its starting position on the physical board. Before the SwiftBot starts moving, it should indicate to the user which square it is moving to.

To visualise the game, the appropriate playing piece should be placed on the corresponding square of the physical board after each move. Players cannot place their pieces on an already occupied square, and they should take turns alternately.

After each turn, the program should display on the console a summary of each player's move. This can be presented as a message as follows or as a visual representation of the board.
[Yasoda – O] moved to square [1,3]
[SwiftBot – X] moved to square [3,3])

Once there are no valid moves left, the program should determine whether the game has ended in a win or a draw. If a player wins, the SwiftBot should trace the winning line. The SwiftBot should blink its lights three times both before and after tracing the winning line; green for an 'O' win and red for an 'X' win. If the game ends in a draw, the SwiftBot should spin once and blink its lights in blue both before and after the move.

After each round, the program should write the following information to a file: the round number, the name of the player, the playing piece (i.e., 'O' or 'X'), the position (i.e. row number and column number) of each piece on the board in the order they were placed, whether the game ended in a win or a draw, the name of the winning player, current date and time.

After a win or a draw, the user should be given the option to play another round or quit. Players can quit the game by pressing the 'X' button on the SwiftBot, and if they wish to continue, they should press the 'Y' button on the robot. The program must also maintain a scoreboard across rounds to track player performance. When a player wins a game, they should be awarded a score.

To move along the grid, the SwiftBot should keep track of its current location and orientation (which direction it is facing and how it should move to reach the next square). You should find a suitable speed for the SwiftBot by experimenting with your robot (refer Calibrate Your SwiftBot worksheet for more guidance).

This program should be implemented as a command-line application, where all input and output are handled via the console. Your implementation must include appropriate error checking and exception handling to ensure that all input values are valid and within the specified range. Invalid inputs should be rejected, and clear error messages should be displayed, prompting the user to re-enter their input. The software design should prioritise usability and user experience.

While the description above outline the required core functionalities, higher grades will be awarded for additional features. These additional functionalities **must not replace or conflict** with the required functionalities of the task. They should be substantial, improve the usability of your program and clearly demonstrate your software development skills.

A substantial additional functionality should:

- accept an input from the user,
- perform meaningful processing based on that input and
- produce an output directly related to the SwiftBot's movement or behaviour.

You may include as many additional functionalities as you wish, however, implementing one substantial additional functionality is sufficient to achieve an A* grade. You may find it useful to discuss this with your tutor.

Task 8: Search for Light

The purpose of this program is to enable the SwiftBot to autonomously navigate and search for a light source in its environment using the onboard camera as the primary input.

To start the program, the user should press button 'A' on the SwiftBot.

At the start of the program, the SwiftBot should capture an image of its surroundings and process the image to determine the direction it should start moving to find light. The program should convert the image into a pixel matrix, divide it into three sections/ columns representing left, centre, and right directions relative to the SwiftBot's orientation. Your program could calculate the average light intensity of each column and use it as an indicator of light present in each direction. The SwiftBot should move in the direction that has the highest average light intensity. For example, if the average intensity in the left-side column is higher, the robot should move to left. If the average intensity in the centre is higher, the robot should move straight ahead. If the average intensity in the right-side is higher, the robot should move to right.

The SwiftBot should be able to move towards a light source in a dark room or towards the brightest area in an already lit room. Therefore, the program should also measure the existing light intensity in the environment and set it as a threshold value to define what constitutes a brighter area.

Before moving towards the light source, the program should display to the user the average light intensities in each column (left, centre and right), in which direction the SwiftBot will move and the speed of the robot. Then the SwiftBot should set the underlights to green, and move forward at a low speed for 1 seconds. To move left or right,

the SwiftBot should turn 30 degrees from its current position in the respective direction. After moving for one second, the SwiftBot should stop and take another image and decide which direction it should move next.

Additionally, before each movement, the SwiftBot should check for any objects within 50 cm ahead of it. If an object is detected, it should blink the underlights in red to inform the user and display the distance to the object. Then the SwiftBot should continue its light-seeking process while avoiding the obstacle. This would require the SwiftBot to change its direction and move towards the direction with the second highest average light intensity calculated at the beginning of the current light-seeking cycle. For example, let's say at the start of the current light-seeking cycle, the average light intensities were calculated as follows: avg_centre > avg_left > avg_right. If an object is detected in front of the SwiftBot, the SwiftBot should then turn towards the left section, as it had the second highest average light intensity after the centre section. If the remaining two average light intensities are exactly the same, then the SwiftBot should randomly select a direction and move. This allows the SwiftBot to continue its light-seeking process while avoiding the obstacle. When an object is encountered, the SwiftBot should take an image of the object and store it as well.

While searching for light, if the SwiftBot has not encountered a light source (i.e. average light intensity remains unchanged), it should wander around (move in a randomly chosen direction) for 1 second and resume the light-seeking process.

If the SwiftBot detects 5 objects in less than 5 minutes, the program should prompt the user to terminate the program remotely. To terminate the program, the user should enter the command 'TERMINATE'. This is a case sensitive command.

Before the program terminates, it should write the log information to a text file. The program should write the following information: the light intensity at the start of the journey (i.e. the threshold light intensity), the brightest light source detected (i.e. the highest average intensity observed), the number of times the SwiftBot detected light and the intensity of light at each instance, the duration of the execution, total distance travelled, movements of the SwiftBot for the whole duration of the journey in the order they were completed (e.g. Straight ahead 15 cm, Left 20 cm, right 30 cm), the number of objects detected and locations which the images and the log file are saved.

This program should be implemented using a command-line user interface (input and output printed in the command prompt). Your program should make appropriate error checks and exception handling in order to ensure that the input values are within the valid range specified in the task description. Invalid input should be rejected, and informative error messages should be displayed, prompting the user for new input. The software should be designed to enhance user experience and ease of use.

While the description above outline the required core functionalities, higher grades will be awarded for additional features. These additional functionalities **must not replace or conflict** with the required functionalities of the task. They should be substantial, improve the usability of your program and clearly demonstrate your software development skills.

A substantial additional functionality should:

- accept an input from the user,
- perform meaningful processing based on that input and
- produce an output directly related to the SwiftBot's movement or behaviour.

You may include as many additional functionalities as you wish, however, implementing one substantial additional functionality is sufficient to achieve an A* grade. You may find it useful to discuss this with your tutor.

Task 9: Dance

The purpose of this program is to make your SwiftBot perform a dance routine in which steps consist of only forward and spinning movements.

The dance is determined by the user input. However, we will make this problem slightly more interesting by
using different number systems. We will be using binary, decimal, octal and hexadecimal number systems. So, your program should also serve as a number converter.

When the program starts it should prompt the user to scan a QR code which contains a hexadecimal number of either 1 or 2 digits (e.g., 'F' or '1F'). The letters in the hexadecimal number should be case insensitive. The program should be able to accept multiple hexadecimal numbers in one QR code. When multiple hexadecimal numbers are provided, the values should be separated by an ampersand (" & "). When multiple values are specified, the SwiftBot will execute them in sequence. For example, when given '1F&2D', the Swiftbot should execute the dance routine for '1F' and then perform a second routine for '2D'. A user can specify only up to five hexadecimal values using one QR code. Before moving, the SwiftBot should verify the values are correct. It should ignore any incorrect values and perform the dance move for only the correct ones. If any value is ignored, the user should be notified of the ignored values before the dance move.

The program should convert this hexadecimal number into the octal, decimal and binary equivalents. For example, if the user enters 5A, the octal equivalent is 132, the decimal equivalent is 90 and the binary equivalent is 1011010.

These converted numbers determine the dance routine parameters.

The speed of the SwiftBot is determined by the octal equivalent of the given hexadecimal number. However, if the octal number is smaller than 50 then the speed should be set to that octal number + 50. If the octal number exceeds the speed limit of the wheels of the SwiftBot, the speed should be set to the maximum limit of SwiftBot.

The colour of the underlights of the SwiftBot is created by mixing different values of red, green, and blue. The red value should be the decimal equivalent of the given hexadecimal number; the green value should be equal to the remainder of the decimal equivalent when divided by 80, multiplied by 3; the blue value should be equal to the greater of the two (red or green) values.

The movements of the robot (forward or spin) are determined by the binary equivalent of the given hexadecimal number. Specifically, the movements should be as follows: reading

the binary number from right to left one digit at a time, the SwiftBot should move forward when the digit is equal to 1, and spin when the digit is equal to zero. So, for example, 1011010 should make the robot to spin, move forward, spin, move forward, move forward, spin and finally move forward. If the hexadecimal is one digit long, the duration of forward movement should be 1 second. If the hexadecimal is two digits long, the duration of forward movement should be 0.5 seconds.

Before the SwiftBot starts moving, the program should display the following information: Hexadecimal value given, the octal, decimal and binary equivalents, the speed at which the SwiftBot robot will be moving, and the red, green, and blue values that make the LED colour.

Here are three examples of output:
F, 17, 15, 1111, speed = 67, LED colour (red 15, green 45, blue 45).
3F, 77, 63, 111111, speed = 77, LED colour (red 63, green 189, blue 189).
C8, 310, 200, 11001000, speed = 100, LED colour (red 200, green 120, blue 200).

The underlights in the SwiftBot should be set to the right colour and the SwiftBot should start moving.

When the SwiftBot completes its last move, it should switch off the LED, and ask the user if they would like to enter another hex number. The user may respond yes by pressing the 'Y' button on the SwiftBot and the program should prompt the user to scan another QR code. If the user responds 'NO' by pressing 'X' button on the SwiftBot, the program should sort all the hexadecimals that the user has entered so far in ascending order, write the sorted numbers to a text file, and display the file path the log file is saved, before terminating the program.

The program must incorporate a 'proper' algorithm for doing the conversion and must NOT use any Java built-in methods to do the conversion.

This program should be implemented using a command-line user interface (input and output printed in the command prompt). Your program should make appropriate error checks and exception handling in order to ensure that the input values are within the valid range specified in the task description. Invalid input should be rejected, and informative error messages should be displayed, prompting the user for new input. The software should be designed to enhance user experience and ease of use.

While the description above outline the required core functionalities, higher grades will be awarded for additional features. These additional functionalities **must not replace or conflict** with the required functionalities of the task. They should be substantial, improve the usability of your program and clearly demonstrate your software development skills.

A substantial additional functionality should:

- accept an input from the user,
- perform meaningful processing based on that input and
- produce an output directly related to the SwiftBot's movement or behaviour.

You may include as many additional functionalities as you wish, however, implementing one substantial additional functionality is sufficient to achieve an A* grade. You may find it useful to discuss this with your tutor.

For testing purposes, you may use any free QR code generator (such as https://qr.io) to create text-based QR codes.

Task 10: Detect Object

The purpose of this program is to make the SwiftBot survey a space either by following or avoiding an object within a specified range. Before surveying an area, each robot is assigned one of the three roles; 'Curious SwiftBot' or 'Scaredy SwiftBot' or 'Dubious SwiftBot'.

When the program starts, it should prompt the user to specify a mode by scanning a QR code containing one of these three mode names: 'Curious SwiftBot', 'Scaredy SwiftBot' or 'Dubious SwiftBot'. The mode names should be encoded exactly as mentioned above.

Then the SwiftBot should start wandering around at a low speed until it encounters an object. When wandering around the underlights should be set to blue.

In 'Curious SwiftBot' mode, the underlights should turn green, when it encounters an object. The SwiftBot should always maintain a gap (buffer zone) of 30 cm between the SwiftBot and the object. If the object is placed beyond the buffer zone, the SwiftBot should turn the underlights to green and start moving towards the object, and stop once it reaches the required gap from the object and turn off the lights. If the object is placed inside the buffer zone, SwiftBot should turn the underlights to green and move backwards to form the required gap, then stop and turn off the underlights. If the object is placed at the required gap, SwiftBot should blink the underglights in green and remain stationary. Once the robot completes a movement (moving backward/ forward/ remaining stationary) it should take an image of the object in front and save it. Then wait for five seconds and check whether the object has moved (backwards or forwards) from its last position and move to maintain the required gap. If the SwiftBot has not encountered an object for five seconds or the object has not moved for five seconds, it should wait for a second, and start moving again in a slightly different direction.

In 'Scaredy SwiftBot' mode, when the SwiftBot encounters an object within 50 cm of its current position it should do the following: take an image of the object and save it, and then blink the underlights, back up and turn in the opposite direction and move away from it for three seconds. The underlights should be set to blue when the SwiftBot is wandering around, and it should change to red when the SwiftBot sees an object and is moving away from it. If the SwiftBot has not encountered an object for five seconds, it should wait for a second, and start moving again in a slightly different direction. The object should be placed in front of the SwiftBot and should move only backwards or forwards, from where it was originally placed. The object shouldn't move sideways.

In 'Dubious SwiftBot' mode, the program should randomly choose and execute either the 'Curious' or the 'Scaredy' mode.

The appropriate speeds for the SwiftBot can be calculated through experimentation with your SwiftBot using simple mathematics (refer Calibrate Your SwiftBot worksheet for more guidance).

Once the SwiftBot encounters more than 3 objects in less than 5 minutes in the selected mode, the program should prompt the user to either change the mode or terminate the program. The program should stop when the user presses the 'X' button on the SwiftBot.

Before termination, the program should write the following information to a log file: the mode it ran and for each mode; the duration of execution, number of times the SwiftBot encountered an object, file path where the images are saved. Then display the file path where the log file is saved to the user.

This program should be implemented using a command-line user interface (input and output printed in the command prompt). Your program should make appropriate error checks and exception handling in order to ensure that the input values are within the valid range specified in the task description. Invalid input should be rejected, and informative error messages should be displayed, prompting the user for new input. The software should be designed to enhance user experience and ease of use.

While the description above outline the required core functionalities, higher grades will be awarded for additional features. These additional functionalities **must not replace or conflict** with the required functionalities of the task. They should be substantial, improve the usability of your program and clearly demonstrate your software development skills.

A substantial additional functionality should:

- accept an input from the user,
- perform meaningful processing based on that input and
- produce an output directly related to the SwiftBot's movement or behaviour.

You may include as many additional functionalities as you wish, however, implementing one substantial additional functionality is sufficient to achieve an A* grade. You may find it useful to discuss this with your tutor.

For testing purposes, you may use any free QR code generator (such as https://qr.io) to create text-based QR codes.

Task 11: Tunnel Vision (for groups with 11 active members)

The purpose of this program is to make your SwiftBot measure distance using the camera as input.

For this task, at least three tunnels of different lengths should be used. The minimum length of a tunnel should be 30 cm and the lengths should vary from about 10 - 15 cm from one another. For example, lengths of the three tunnels could be 45 cm, 57 cm and 67 cm. A tunnel should be wide enough for the SwiftBot to pass through and all the tunnels should have the same height and width. The three tunnels should be placed one after the other, in any order, forming a roadway. Leave sufficient gaps between consecutive tunnels to detect the change in lighting which will be used as an indication of the beginning and end

of a tunnel. You will need to create your own set of tunnels for testing your program. During the examination, we will provide the tunnels for you.

To start the program, the user should enter the command 'START'. This is a case sensitive command.

Then, the SwiftBot's underlights should be set to red and the robot should start moving forward at a slow speed. You need to use light intensity as an indicator of SwiftBot's location. For example, the light intensity inside the tunnel is lower compared to the outside. Therefore, the surrounding light intensity can be used to determine whether the SwiftBot is inside or outside the tunnel. To track these changes, the SwiftBot should start taking images at regular intervals as soon as it starts moving. Your program should process the images to determine the light intensity. For example, the program could convert an image into a pixel matrix and use the pixel values to compute the average light intensity at a particular location. Then check for changes in average light intensity to detect when the robot enters and leaves the tunnel. You can then calculate the distance that the SwiftBot moved using simple mathematics (refer Calibrate Your SwiftBot worksheet for more guidance). You should find a suitable speed for the SwiftBot, a frequency for taking images and gap between tunnels by experimenting with your robot.

If the SwiftBot detects an object within 40 cm of its path, it should stop. Then take a photo of the object, inform the user about the obstacle on course. At this point, the user should be prompted to recall the robot by entering the command 'RETURN'. This is a case-sensitive command. Upon receiving this command, the SwiftBot should return to the starting position of the tunnel by moving backwards and then terminate the program.

After every three tunnels detected (i.e., after the 3rd, 6th, 9th, and so on), the program should prompt the user to either recall the robot or continue.

Before the program terminates, it should write the following information to a log file: the total number of tunnels encountered, length of each tunnel, average light intensity inside a tunnel, the distance between two consecutive tunnels, total distance travelled, the duration of the execution, whether an obstacle was detected and if so the file path where the photo is saved. The program should display the file path where the log file is saved to the user.

This program should be implemented using a command-line user interface (input and output printed in the command prompt). Your program should make appropriate error checks and exception handling in order to ensure that the input values are within the valid range specified in the task description. Invalid input should be rejected, and informative error messages should be displayed, prompting the user for new input. The software should be designed to enhance user experience and ease of use.

While the description above outline the required core functionalities, higher grades will be awarded for additional features. These additional functionalities **must not replace or conflict** with the required functionalities of the task. They should be substantial, improve the usability of your program and clearly demonstrate your software development skills.

A substantial additional functionality should:

- accept an input from the user,
- perform meaningful processing based on that input and
- produce an output directly related to the SwiftBot's movement or behaviour.

You may include as many additional functionalities as you wish, however, implementing one substantial additional functionality is sufficient to achieve an A* grade. You may find it useful to discuss this with your tutor.